

A 28-nm Real-Time Reinforcement Learning Processor for Mapless Autonomous Navigation with Unified Actor-Critic Network and Inference-on-Request Scheduling

Juyoung Oh¹, Jie-Xin Liu², Yi-Chen Teng², Hsueh-Cheng Wang², Dongsuk Jeon¹

¹Seoul National University, Seoul, Korea

²National Yang Ming Chiao Tung University, Hsinchu, Taiwan

Autonomous navigation represents the task of navigating between points without human intervention. In many real-world scenarios, the maps are unavailable or unreliable, necessitating robust mapless navigation techniques that rely solely on local observations and target position information without access to global map data [1]. Deep Reinforcement Learning (DRL) has emerged as a viable solution for complex tasks such as motion control [2], game playing [3], and autonomous driving [4], and it is particularly effective for mapless navigation since it allows for systems to adapt to new environments without relying on predefined maps [5]. Traditional DRL approaches such as the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm [6] use scalar Q values to represent the expected reward for specific observation-action pairs [7]. However, those Q values may not effectively capture the uncertainty of the value function, undermining reliability when applied to autonomous navigation [8]. Contrarily, distributional reinforcement learning such as the Distributed Distributional Deep Deterministic Policy Gradient (D4PG) algorithm [9], which models Q values as distributions, provides a more comprehensive understanding of reward structures and is commonly employed in navigation tasks due to its superior performance [10-12].

Deploying DRL algorithms including D4PG on real robots presents critical challenges, as shown in Fig. 1. DRL typically relies on multiple neural networks (e.g., four in D4PG), often comprising fully connected layers, which demand substantial external memory access (EMA) and computational resources. Comprehensive system optimization, from algorithm to hardware, is essential for maximizing processing efficiency by fully utilizing available external memory bandwidth and on-chip computing circuits. This optimization can be achieved by identifying an optimal design point in the roofline model (Fig. 1, center right) [13], where arithmetic intensity represents the ratio of computational capability to memory bandwidth for hardware and the ratio of total operations to the amount of required data for an algorithm. In addition, mapless autonomous navigation demands incremental learning to adapt to dynamic environments, requiring the processor to efficiently support concurrent training and inference. Since a robot's sensors typically operate at a fixed sampling frequency, inference needs to be performed in sync with this interval. Contrarily, training is required much less frequently than inference, as sparse training (i.e., training conducted only after a set number of inferences) demonstrates significantly higher performance [14].

To address these challenges, this paper presents an energy-efficient real-time reinforcement learning processor aimed at mapless autonomous navigation. Our main contributions are: 1) a unified actor-critic network architecture that reduces EMA and computational costs, 2) an Inference-on-Request (IoR) scheduling scheme for real-time robot control with minimal energy cost, 3) adaptive dataflow mapping that efficiently supports both single- and multi-batch processing, and 4) an energy-efficient column-wise zero-skipping categorical projection, all contributing to an optimal processor design with an algorithm-matched arithmetic intensity that ensures maximum utilization of hardware resources. Based on these contributions, the proposed design realizes real-time autonomous navigation in sync with the robot's control rate, consuming only 2.68 mW in total.

Fig. 2 illustrates the proposed unified actor-critic network architecture. In actor-critic-based algorithms including D4PG, two distinct neural networks are employed: the actor network generates actions based on the observations, while the critic network evaluates the effectiveness of these actions in the current observation. Due to the structural similarity between these networks, we propose to employ a unified neural network architecture for actor and critic networks, where most convolutional (Conv) and fully connected (FC) layers are shared except for four non-shared FC layers to capture the distinct characteristics of each operation, significantly reducing parameters, EMA, and operations. During training, the actor and critic networks should be executed sequentially, both taking identical observations as input. Our design minimizes computational overheads by caching the intermediate feature map from the actor network for later use in the critic network, avoiding duplicate computations (Fig. 2, center right). In simulations, our approach reduces EMA and operation count by 84.1% and 85.4%, respectively, during training, while maintaining network performance compared to the conventional separate actor-critic network (Fig. 2, bottom).

Sparse training enables reliable training convergence and higher performance. Simulations show that an inference-to-training ratio (N) of 8 reduces the required training iterations by $2.27\times$ compared to conventional dense training (Fig. 3, center left). For sparse training, conventional sequential processing of inference and training employed in prior RL accelerators [16-18] suffers from unavoidable hardware idle time and requires a higher-than-necessary operating frequency (Fig. 3, top). Our design addresses this challenge with an Inference-on-Request (IoR) scheduling scheme. In this approach, the computationally intensive training process runs in the background, while inference is prioritized and triggered as needed by the robot. Upon receiving a new sensor input, the system interrupts ongoing training and immediately starts inference, meeting control rate constraints without additional power consumption. Simulations confirm that the IoR scheduling achieves full hardware utilization across different inference-to-training ratios, reducing the required minimum operating frequency by 86.4% for $N=8$. IoR scheduling is managed by the Instruction Management Unit (IMU), which monitors training instructions and determines whether they can be interrupted, maximizing on-chip data reuse and avoiding redundant EMA. More specifically, an interrupt is accepted once all operations that can be performed with on-chip stored data are completed and no further reuse of on-chip data is possible. With IoR scheduling, the worst-case latency for inference remains under 10% of the sensor input period, ensuring real-time control of the robot.

Fig. 4 displays the overall architecture of the proposed reinforcement learning processor. It consists of a PE array with 8×32 processing elements (PEs) that support BF16 MAC operations with FP32 accumulation, 16-kB Multi/Broadcasting Memory (MBMEM), 512-kB Multi/Unicasting Memory (MUMEM), 64-kB output memory, a Vector Processing Unit (VPU), and a Projection Processing Unit (PPU). The design employs 256 PEs to match the arithmetic intensity of the processor with that of the neural network model (Fig. 4, top right). As a result, the intensity ratio remains close to 1 throughout the entire navigation, avoiding memory or compute bottlenecks encountered in previous designs. In addition, the processor adopts an adder-tree-less output stationary architecture to minimize the overhead associated with 32-bit partial sum accumulation. VPU handles element-wise operations, including non-linear activation functions, while PPU manages categorical projection that is essential for distributional RL including D4PG. MBMEM and MUMEM distribute data across the PE array differently depending on the batch size. For multi-batch training, weights are stored in MBMEM and multicasted row-wise, while activations are stored in MUMEM and multicasted column-wise. For single-batch inference, activations are stored in MBMEM and broadcasted to all processing elements, while weights are stored in MUMEM and unicasted to each processing element. This adaptive dataflow mapping achieves a high PE utilization of 71.0% during concurrent inference and training, including I/O latency.

Distributional RL including D4PG utilizes Q-value distributions instead of scalar values to better capture the uncertainty of the value function. Categorical projection is essential in distributional RL to align the target Q-value distribution with the original distribution by redistributing probabilities across bins, accounting for the changes in their values caused by the discount factor (γ) and reward (R). Our design optimizes categorical projection by leveraging its characteristics to reduce computational complexity and increase throughput (Fig. 5). The Δh matrix, which represents the probability transfer from source to destination bins, is sparse, with at most two neighboring non-zero values in each column. Our design leverages this property to improve energy efficiency by processing one column of the Δh matrix per cycle, focusing only on the non-zero values. This column-wise zero-skipping mechanism reduces the required operations by 99% while double-buffering in PPU maintains high throughput.

Fig. 6 displays measurement results and comparisons with state-of-the-art designs. Fabricated in 28-nm CMOS, the processor occupies $1.53\times 1.53\text{mm}^2$ and consumes 2.68 mW at 0.67 V and 10 MHz, successfully meeting the control rate constraint of the robot. Compared to prior designs [16-18], our processor achieves the highest EMA reduction of 84.1% and PE utilization of 71.0% during concurrent inference and training including I/O latency. The design is the first RL processor to support distributional reinforcement learning (D4PG), significantly outperforming the conventional RL algorithm (TD3) adopted in prior designs (Fig. 6, top). In addition, the processor was successfully verified on a real mobile robot for mapless autonomous navigation in an indoor environment with multiple obstacles, demonstrating its capability of adapting to unseen environments. Fig. 7 shows the die micrograph.

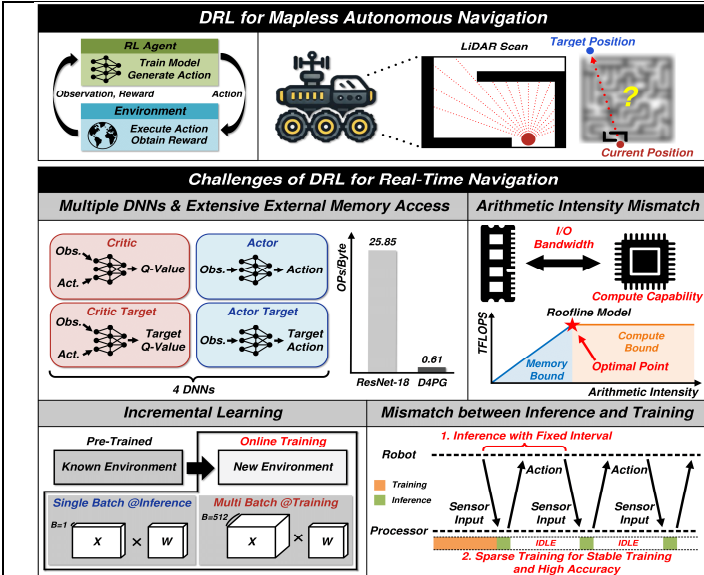


Fig. 1. Overview and challenges of deep reinforcement learning for mapless autonomous navigation.

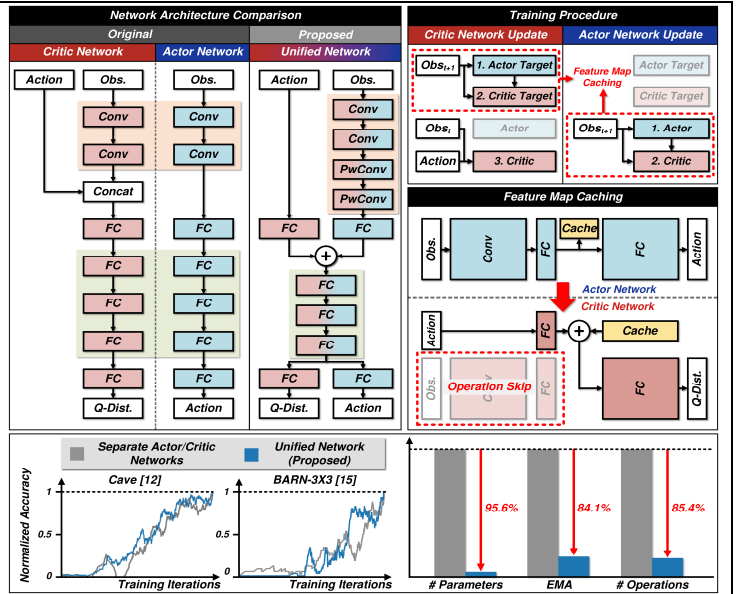


Fig. 2. Proposed unified actor-critic network architecture.

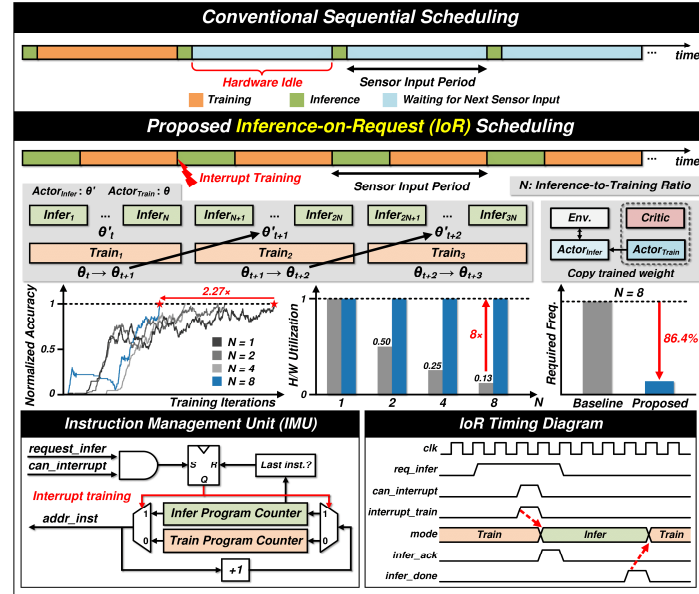


Fig. 3. Proposed Inference-on-Request (IoR) scheduling.

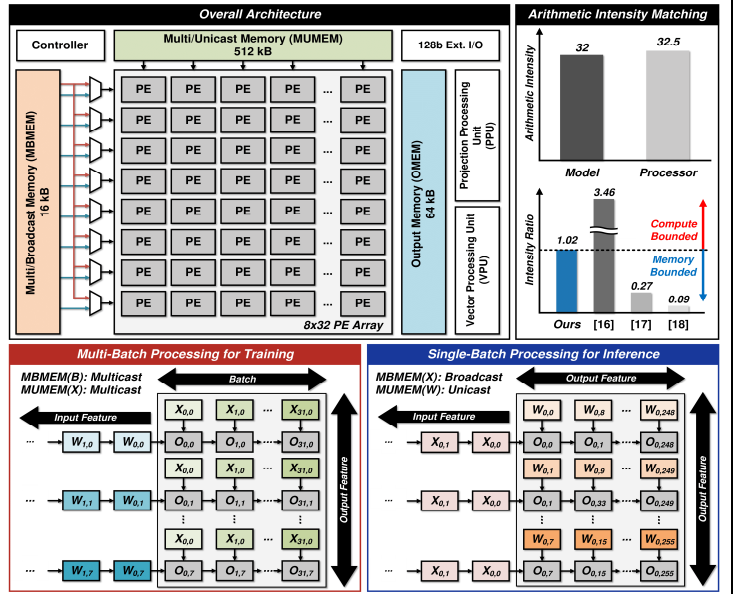


Fig. 4. Overall architecture and batch-adaptive dataflow.

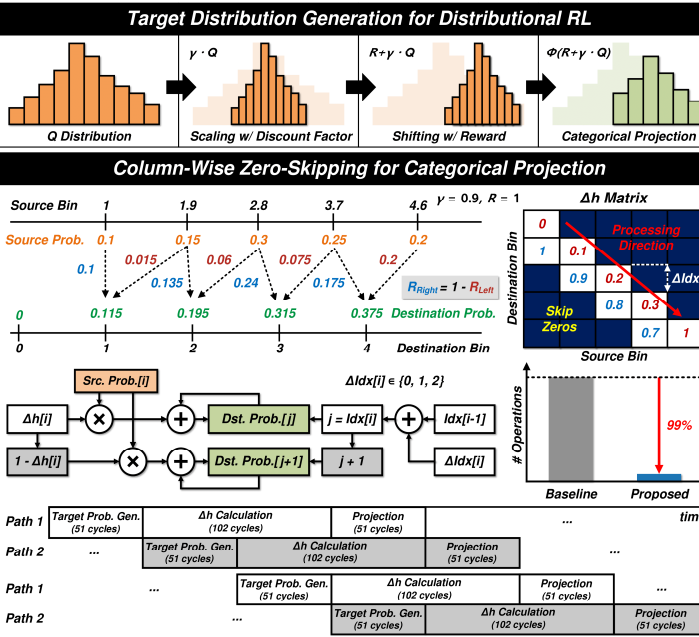


Fig. 5. Proposed column-wise zero-skipping categorical projection.

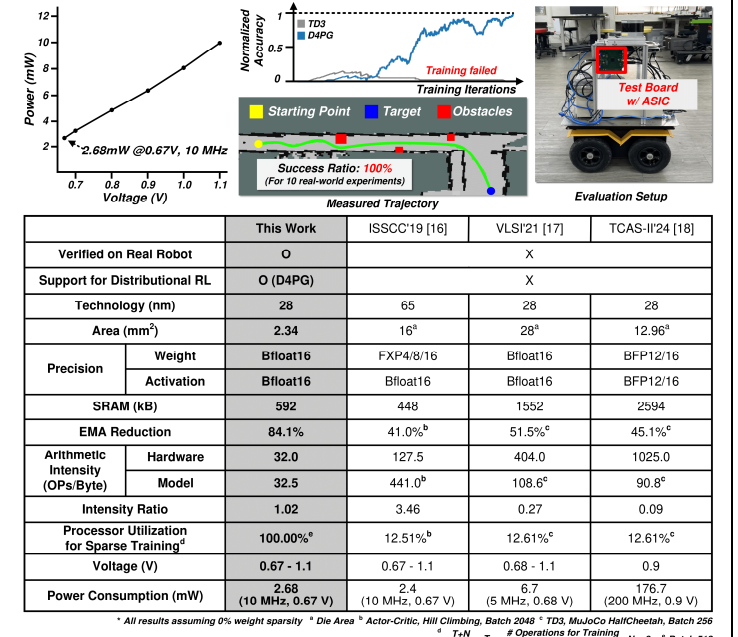


Fig. 6. Measurement results and comparisons with prior work.

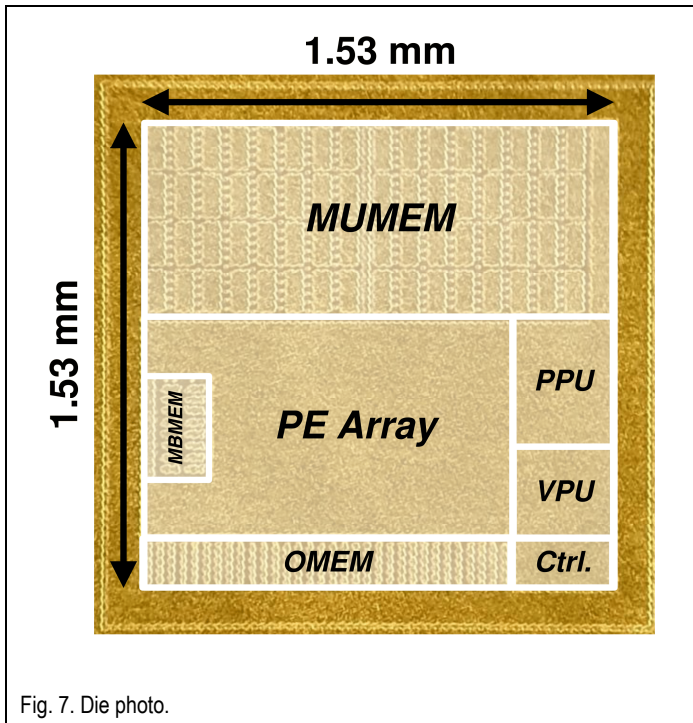


Fig. 7. Die photo.

Acknowledgements

This work was supported by the National Research Foundation of Korea (Grant NRF-2022R1C1C1006880). The chip fabrication and EDA tool were supported by the IC Design Education Center.

References

- [1] L. Tai et al, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation", IROS, 2017. <https://doi.org/10.1109/IROS.2017.8202134>
- [2] P. Lillicrap et al, "Continuous control with deep reinforcement learning", ICLR, 2016. <https://doi.org/10.48550/arXiv.1509.02971>.
- [3] V. Mnih et al, "Playing Atari with Deep Reinforcement Learning", ArXiv, 2013. <https://doi.org/10.48550/arXiv.1312.5602>.
- [4] A. Kendall et al, "Learning to Drive in a Day", ICRA, 2019. <https://doi.org/10.1109/ICRA.2019.8793742>.
- [5] R. B. Grando et al, "Deep Reinforcement Learning for Mapless Navigation of a Hybrid Aerial Underwater Vehicle with Medium Transition", ICRA, 2021. <https://doi.org/10.1109/ICRA48506.2021.9561188>.
- [6] S. Fujimoto et al, "Addressing Function Approximation Error in Actor-Critic Methods", PMLR, 2018. <https://doi.org/10.48550/arXiv.1802.09477>.
- [7] X. Lin et al, "Robust Unmanned Surface Vehicle Navigation with Distributional Reinforcement Learning", IROS, 2023. <https://doi.org/10.1109/IROS55552.2023.10342389>.
- [8] X. Lin et al, "Robust Route Planning with Distributional Reinforcement Learning in a Stochastic Road Network Environment", UR, 2023. <https://doi.org/10.1109/UR57808.2023.10202222>.
- [9] G. Barth-Maron et al, "Distributed Distributional Deterministic Policy Gradients", ICLR, 2018. <https://doi.org/10.48550/arXiv.1804.08617>.
- [10] C. Lyle et al, "A comparative analysis of expected and distributional reinforcement learning", AAAI, 2019. <https://doi.org/10.1609/aaai.v33i01.33014504>
- [11] L. Chen-Lung et al, "A Heterogeneous Unmanned Ground Vehicle and Blimp Robot Team for Search and Rescue using Data-driven Autonomy and Communication-aware Navigation", Field Robotics, 2022. <https://doi.org/10.55417/fr.2022020>.
- [12] H. -C. Wang et al, "Curriculum Reinforcement Learning From Avoiding Collisions to Navigating Among Movable Obstacles in Diverse Environments", RA-L, 2023. <https://doi.org/10.1109/LRA.2023.3251193>.
- [13] S. Williams et al, "Roofline: an insightful visual performance model for multicore architectures", CACM, 2009. <https://doi.org/10.1145/1498765.1498785>.
- [14] W. Fedus et al, "Revisiting Fundamentals of Experience Replay", PMLR, 2020. <https://doi.org/10.48550/arXiv.2007.06700>.
- [15] V. Orekhov et al, "The DARPA Subterranean Challenge: A Synopsis of the Circuits Stage", Field Robotics, 2022. <https://doi.org/10.55417/fr.2022024>
- [16] C. Kim et al, "A 2.1TFLOPS/W Mobile Deep RL Accelerator with Transposable PE Array and Experience Compression", ISSCC, 2019. <https://doi.org/10.1109/ISSCC.2019.8662447>.
- [17] J. Lee et al, "OmniDRL: A 29.3 TFLOPS/W Deep Reinforcement Learning Processor with Dualmode Weight Compression and On-chip Sparse Weight Transposer", VLSI, 2021. <https://doi.org/10.23919/VLSICircuits52068.2021.9492504>.
- [18] S. An et al, "A 8.81 TFLOPS/W Deep-Reinforcement-Learning Accelerator With Delta-Based Weight Sharing and Block-Mantissa Reconfigurable PE Array", TCAS-II, 2024. <https://doi.org/10.1109/TCSSII.2024.3374725>.